Amazon DynamoDB

Trabalho acadêmico do Grupo G3 | Pesquisa e tutorial de uso

O Amazon DynamoDB é um banco de dados não relacional (NoSQL), distribuído e gerenciado pela Amazon Web Services (AWS). Ele foi projetado para aplicações que precisam armazenar e consultar dados com baixa latência, alta disponibilidade e capacidade de crescimento. Este trabalho apresenta seus recursos, sua evolução, exemplos de uso e uma demonstração com comandos. [1]

1. Principais características

Modelo chave-valor e documentos. Armazena dados em tabelas, compostas por itens e atributos. Um item representa um registro; seus atributos representam informações como nome e idade. Também admite listas e mapas aninhados. [2]

Chave primária. Cada item é identificado por uma chave de partição. A tabela pode ter também uma chave de ordenação; nesse caso, a combinação das duas identifica o item. A escolha das chaves influencia a distribuição dos dados e as consultas. [2]

Estrutura flexível. Os atributos que não compõem as chaves podem variar entre os itens. Isso permite armazenar registros diferentes na mesma tabela, mas ainda exige planejamento da modelagem. [2, 7]

Serviço gerenciado e serverless. A AWS administra a infraestrutura do banco. A aplicação utiliza o serviço sem instalar ou manter servidores de banco de dados. [1]

Escalabilidade e desempenho. É voltado a operações frequentes de leitura e escrita com resposta em milissegundos. O desempenho depende do padrão de acesso, das chaves escolhidas e da configuração de capacidade. [1, 7]

Capacidade e cobrança. Oferece os modos sob demanda e provisionado. No primeiro, a cobrança acompanha as solicitações; no segundo, a capacidade é definida antecipadamente, com possibilidade de ajuste automático. [12, 13]

Disponibilidade e segurança. Replica dados entre zonas de disponibilidade de uma região. Oferece criptografia em repouso, controle de acesso pelo IAM e recursos de backup e recuperação. [12]

Consistência e transações. Oferece leituras eventualmente consistentes e, nas operações compatíveis, fortemente consistentes. Índices secundários globais não oferecem leitura forte. Suporta transações ACID: atomicidade, consistência, isolamento e durabilidade. [3, 4]

Índices e integração. Índices secundários permitem outros caminhos de consulta. O DynamoDB Streams registra alterações nos itens e pode acionar funções do AWS Lambda. [12]

Histórico e uso por empresas

2. Histórico de desenvolvimento

O DynamoDB surgiu da experiência da Amazon com sistemas de comércio eletrônico em grande escala. A empresa precisava combinar disponibilidade e desempenho com menor esforço de administração. O sistema interno Dynamo influenciou esse desenvolvimento, mas Dynamo e DynamoDB são sistemas distintos. [5]

2007. Publicação do artigo sobre o Dynamo, sistema de armazenamento chave-valor desenvolvido pela Amazon. [5]

18 de janeiro de 2012. Lançamento público do Amazon DynamoDB como serviço gerenciado da AWS. [5]

2013 e 2014. Ampliação dos recursos de índices secundários e introdução do DynamoDB Streams para registrar mudanças nos dados. [5]

2017. Introdução das tabelas globais e de recursos de backup e restauração. [5]

2018. Disponibilização da recuperação para um ponto no tempo e de transações ACID. [5]

Desde então, a AWS continua ampliando o serviço. Sua evolução mantém o foco na administração simplificada, na escala e no acesso rápido aos dados. [1]

3. Utilização por empresas

Os casos abaixo estão documentados na página oficial de clientes do DynamoDB. São exemplos de usos específicos dentro dessas empresas, que também podem utilizar outros bancos de dados. [6]

Disney+. Utiliza o DynamoDB para conteúdo, metadados e ações dos usuários. Os usos incluem listas de conteúdos para assistir, marcações que permitem continuar um vídeo em outro dispositivo e informações relacionadas às recomendações. [6]

Samsung Cloud. Emprega o DynamoDB como índice de metadados de dados de dispositivos Galaxy, como notas, contatos e gravações. Os arquivos ficam no Amazon S3; o DynamoDB organiza informações que ajudam a localizá-los. [6]

Mercado Livre. Adotou o DynamoDB para reduzir o trabalho operacional e sustentar grande escala. O caso destaca a evolução da plataforma com índices secundários e recursos de segurança. [6]

Esses exemplos mostram seu uso em streaming, sincronização de dados e plataformas digitais que precisam crescer sem administrar diretamente servidores de banco de dados.

Vantagens e limitações

4. Quando é vantajoso em relação a um banco relacional?

A vantagem depende das necessidades da aplicação. O DynamoDB é especialmente adequado quando as consultas mais importantes podem ser definidas antecipadamente e atendidas por chaves ou índices. [7]

Grande volume e acesso previsível. É uma opção para aplicações com muitas leituras e escritas, como acesso ao perfil por identificador ou consulta de pedidos de um cliente. A modelagem deve atender a essas operações diretamente. [7]

Tráfego variável. O modo sob demanda permite ajustar a capacidade ao uso, o que pode facilitar a operação em campanhas, lançamentos e outros períodos de aumento de acesso. [12]

Menor esforço de administração. Equipes podem concentrar-se na aplicação enquanto a AWS gerencia a infraestrutura, a manutenção e a disponibilidade do serviço. [1]

Aplicações distribuídas. Tabelas globais permitem replicar dados entre regiões da AWS, aproximando o acesso dos usuários e apoiando estratégias de continuidade. [12]

Dados com atributos variáveis. Itens podem ter atributos diferentes, o que facilita mudanças em registros quando a aplicação tem um modelo de acesso bem planejado. [2, 7]

Exemplo: em uma loja virtual, consultar o carrinho de um usuário por seu identificador é um acesso previsível. O DynamoDB pode ser vantajoso se a loja precisar de grande escala e baixa sobrecarga operacional. Para consultas exploratórias que cruzem muitas entidades, um banco relacional pode ser mais adequado. [7]

5. Situações em que sua utilização é desaconselhável

Muitas junções e relatórios livres. Não oferece o operador JOIN. Consultas complexas entre várias entidades podem exigir desnormalização, índices ou processamento adicional na aplicação. [1]

Consultas imprevisíveis. Se os critérios de busca mudam constantemente, a modelagem pode exigir novos índices ou varreduras. Um Scan lê itens da tabela, mesmo quando parte deles é descartada por um filtro. [16]

Arquivos ou registros grandes. Cada item tem limite de 400 KB, incluindo nomes e valores de atributos. Imagens, vídeos e documentos grandes devem preferencialmente ficar em armazenamento de objetos, com suas referências no banco. [8]

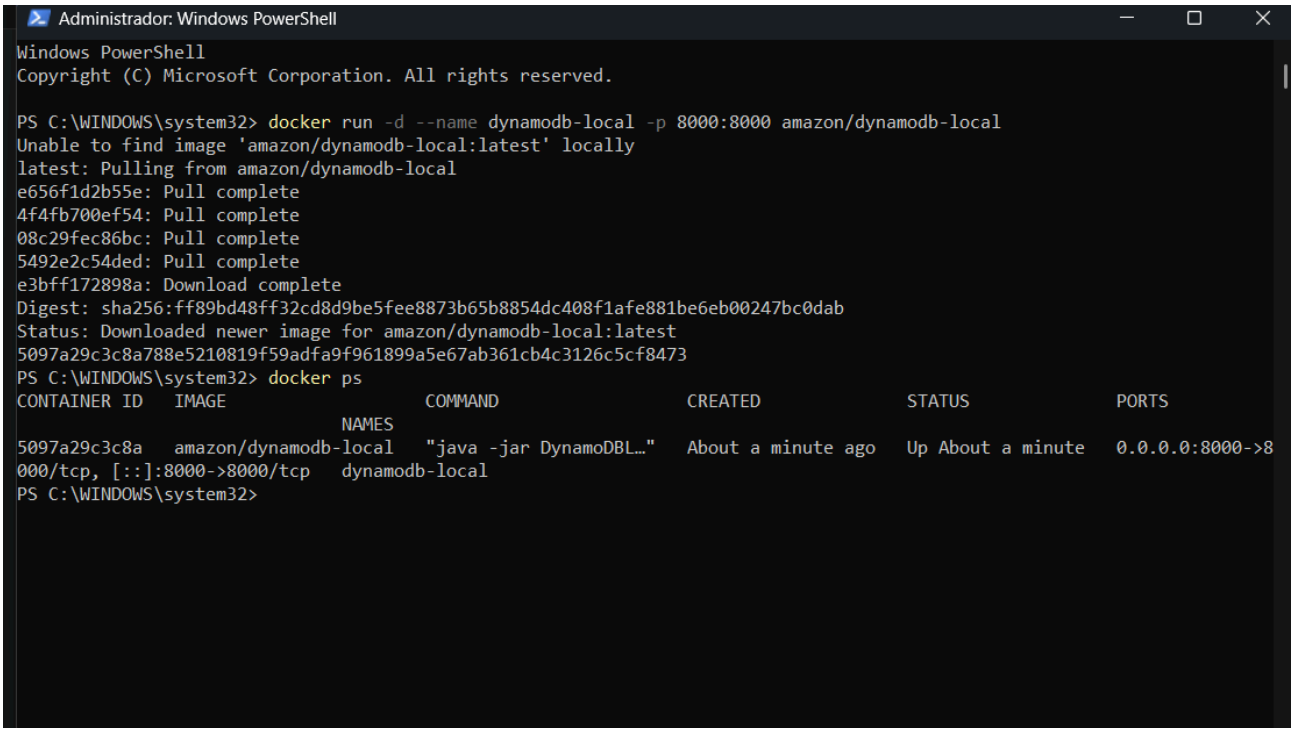
Transações muito extensas. Há suporte ACID, mas uma transação admite até 100 itens distintos e 4 MB. As garantias transacionais não abrangem operações entre regiões. [4, 8]

Dependência de operação local e custos inadequados. O serviço de produção depende da AWS. Se o projeto exige operação independente da nuvem, ou se sua carga resulta em custos desfavoráveis, outras soluções podem atender melhor. [1, 13]

6. Parte prática: DynamoDB Local com Docker

DOCKER:

O comando `docker run` foi utilizado para iniciar o DynamoDB Local em um contêiner Docker. A opção `-d` executa o contêiner em segundo plano, `--name dynamodb-local` define seu nome, `-p 8000:8000` configura a comunicação pelas portas e `amazon/dynamodb-local` indica a imagem utilizada. Dessa forma, conseguimos executar o banco de dados localmente, sem utilizar o serviço na nuvem da AWS.



CRIACAO DA TABELA:

O comando `aws dynamodb create-table` foi utilizado para criar a tabela `AlunosG3DynamoDB`. Definimos `curso` como chave de partição e `matricula` como chave de ordenação, ambas do tipo texto (`S`). Também configuramos o modo `PAY_PER_REQUEST` e utilizamos `--endpoint-url http://localhost:8000` para direcionar a operação ao banco local. O resultado confirmou que a tabela foi criada e estava ativa, inicialmente sem registros.



PS C:\WINDOWS\system32> aws dynamodb create-table --table-name AlunosG3DynamoDB --attribute-definitions AttributeName=curso,AttributeType=S AttributeName=matricula,AttributeType=S --key-schema AttributeName=curso,KeyType=HASH AttributeName=matricula,KeyType=RANGE --billing-mode PAY_PER_REQUEST --endpoint-url http://localhost:8000

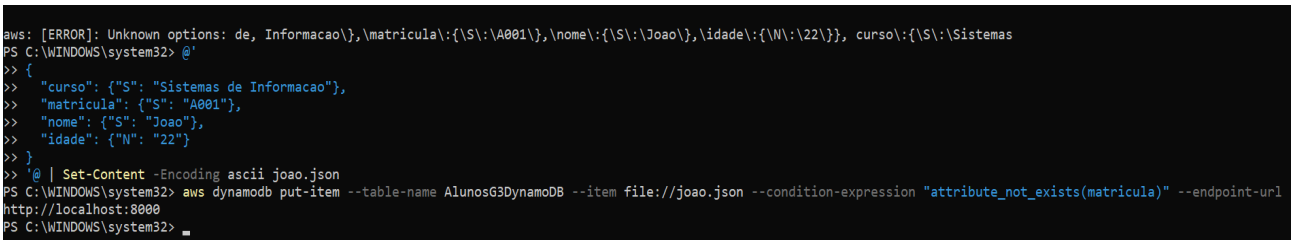
{
"TableDescription": {
"AttributeDefinitions": [
{
"AttributeName": "curso",
"AttributeType": "S"
},
{
"AttributeName": "matricula",
"AttributeType": "S"
}
],
"TableName": "AlunosG3DynamoDB",

```
    "KeySchema": [
        {
            "AttributeName": "curso",
            "KeyType": "HASH"
        },
        {
            "AttributeName": "matricula",
            "KeyType": "RANGE"
        }
    ],
    "TableStatus": "ACTIVE",
    "CreationDateTime": "2026-10-13T15:09:55.660000-03:00",
    "ProvisionedThroughput": {
        "LastIncreaseDateTime": "1969-12-31T21:00:00-03:00",
        "LastDecreaseDateTime": "1969-12-31T21:00:00-03:00",
        "NumberOfDecreasesToday": 0,
        "ReadCapacityUnits": 0,
        "WriteCapacityUnits": 0
    },
    "TableSizeBytes": 0,
    "ItemCount": 0,
    "TableArn": "arn:aws:dynamodb:ddblocal:000000000000:table/AlunosG3DynamoDB",
    "BillingModeSummary": {
        "BillingMode": "PAY_PER_REQUEST",
        "LastUpdateToPayPerRequestDateTime": "2026-10-13T15:09:55.660000-03:00"
    },
    "DeletionProtectionEnabled": false
}
```


-

INSERIR ALUNO NO BANCO DE DADOS:

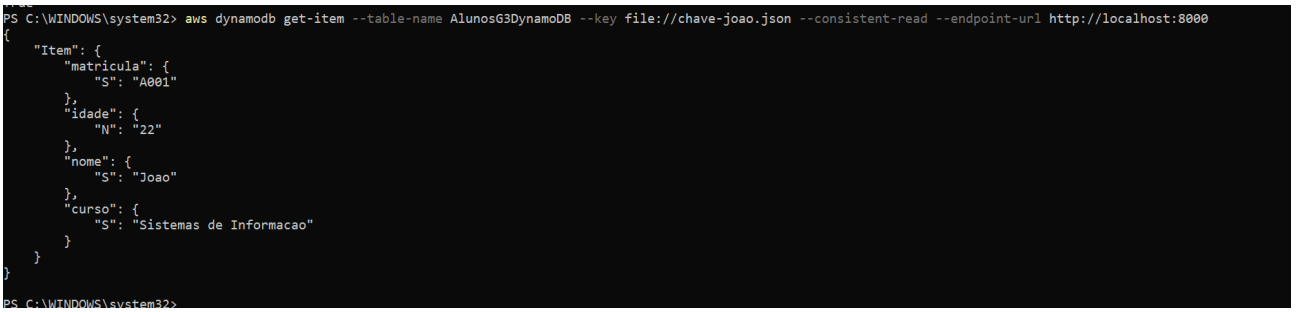
O comando `put-item` foi utilizado para inserir João na tabela. Seus dados foram organizados em um arquivo JSON, contendo curso, matrícula, nome e idade. A opção `--condition-expression "attribute_not_exists(matricula)"` adicionou uma condição para impedir a inserção quando já existisse um item com aquela matrícula na mesma chave de partição.



>> {
>>   "curso": {"S": "Sistemas de Informacao"},
>>   "matricula": {"S": "A001"},
>>   "nome": {"S": "Joao"},
>>   "idade": {"N": "22"}
>> }
>> '@ | Set-Content -Encoding ascii joao.json
PS C:\WINDOWS\system32> aws dynamodb put-item --table-name AlunosG3DynamoDB --item file://joao.json --condition-expression "attribute_not_exists(matricula)" --endpoint-url http://localhost:8000

CONSULTAR ALUNO:

O comando `get-item` foi utilizado para buscar especificamente o registro de João por meio de sua chave primária, informada no arquivo `chave-joao.json`. A opção `--consistent-read` solicitou uma leitura fortemente consistente. O resultado mostrou os dados do aluno armazenados na tabela.

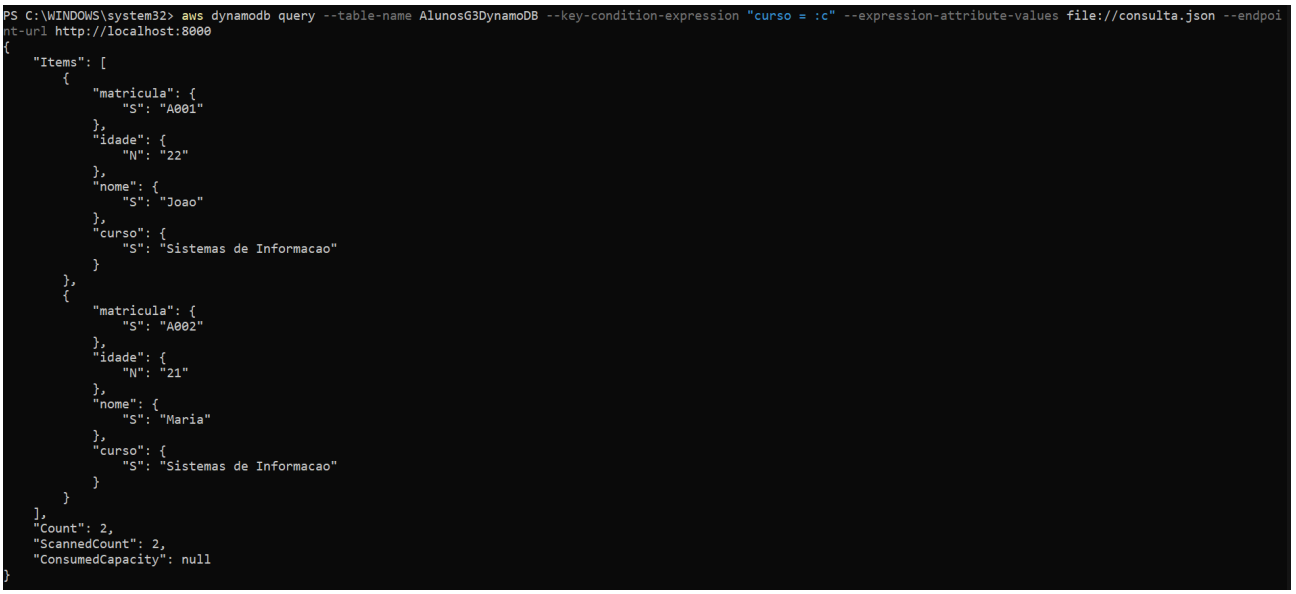


PS C:\WINDOWS\system32> aws dynamodb get-item --table-name AlunosG3DynamoDB --key file://chave-joao.json --consistent-read --endpoint-url http://localhost:8000


```
{
    "Item": {
        "matricula": {
            "S": "A001"
        },
        "idade": {
            "N": "22"
        },
        "nome": {
            "S": "Joao"
        },
        "curso": {
            "S": "Sistemas de Informacao"
        }
    }
}
```


CADASTRANDO OUTRO ALUNO:

Foi inserida Maria na tabela utilizando o comando `put-item`, com matrícula `A002`, nome, idade e curso. A consulta apresentada no resultado mostrou dois registros, João e Maria, confirmando que os dados de ambos estavam armazenados no banco.



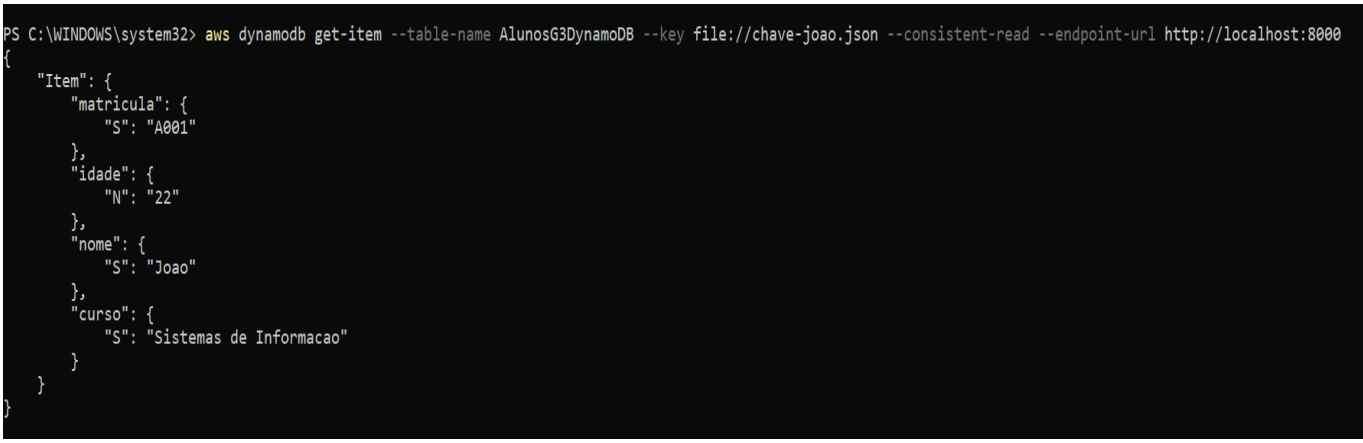
```
PS C:\WINDOWS\system32> aws dynamodb query --table-name AlunosG3DynamoDB --key-condition-expression "curso = :c" --expression-attribute-values file://consulta.json --endpoi
nt-url http://localhost:8000
{
    "Items": [
        {
            "matricula": {
                "S": "A001"
            },
            "idade": {
                "N": "22"
            },
            "nome": {
                "S": "Joao"
            },
            "curso": {
                "S": "Sistemas de Informacao"
            }
        },
        {
            "matricula": {
                "S": "A002"
            },
            "idade": {
                "N": "21"
            },
            "nome": {
                "S": "Maria"
            },
            "curso": {
                "S": "Sistemas de Informacao"
            }
        }
    ],
    "Count": 2,
    "ScannedCount": 2,
    "ConsumedCapacity": null
}
```

nt-url http://localhost:8000
{
    "Items": [
        {
            "matricula": {
                "S": "A001"
            },
            "idade": {
                "N": "22"
            },
            "nome": {
                "S": "Joao"
            },
            "curso": {
                "S": "Sistemas de Informacao"
            }
        },
        {
            "matricula": {
                "S": "A002"
            },
            "idade": {
                "N": "21"
            },
            "nome": {
                "S": "Maria"
            },
            "curso": {

```
            "S": "Sistemas de Informacao"
          }
        }
    ],
    "Count": 2,
    "ScannedCount": 2,
    "ConsumedCapacity": null
}
```


CONSULTANDO ALUNO SEPARADO:

Utilizamos novamente o comando `get-item` para recuperar somente o registro de João. A busca foi realizada pelas chaves informadas no arquivo JSON, demonstrando como localizar um aluno específico sem precisar listar todos os registros da tabela.

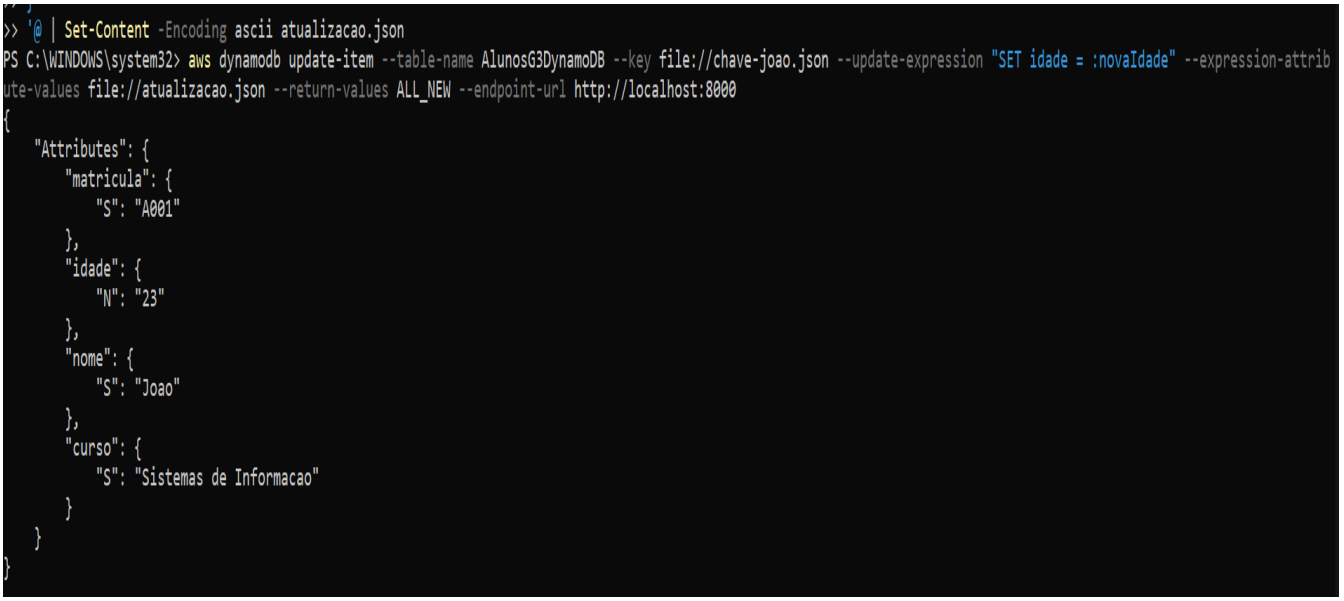


PS C:\WINDOWS\system32> aws dynamodb get-item --table-name AlunosG3DynamoDB --key file://chave-joao.json --consistent-read --endpoint-url http://localhost:8000
{
    "Item": {
        "matricula": {
            "S": "A001"
        },
        "idade": {
            "N": "22"
        },
        "nome": {

```
            "S": "Joao"
        },
        "curso": {
            "S": "Sistemas de Informacao"
        }
    }
}
```


ATUALIZAMOS A IDADE DE JOAO:

O comando `update-item` foi utilizado para alterar a idade de João de 22 para 23 anos. A expressão `SET idade = :novaIdade` definiu o atributo que seria atualizado, enquanto o arquivo `atualizacao.json` forneceu o novo valor. A opção `--return-values ALL_NEW` retornou os dados do item após a atualização.



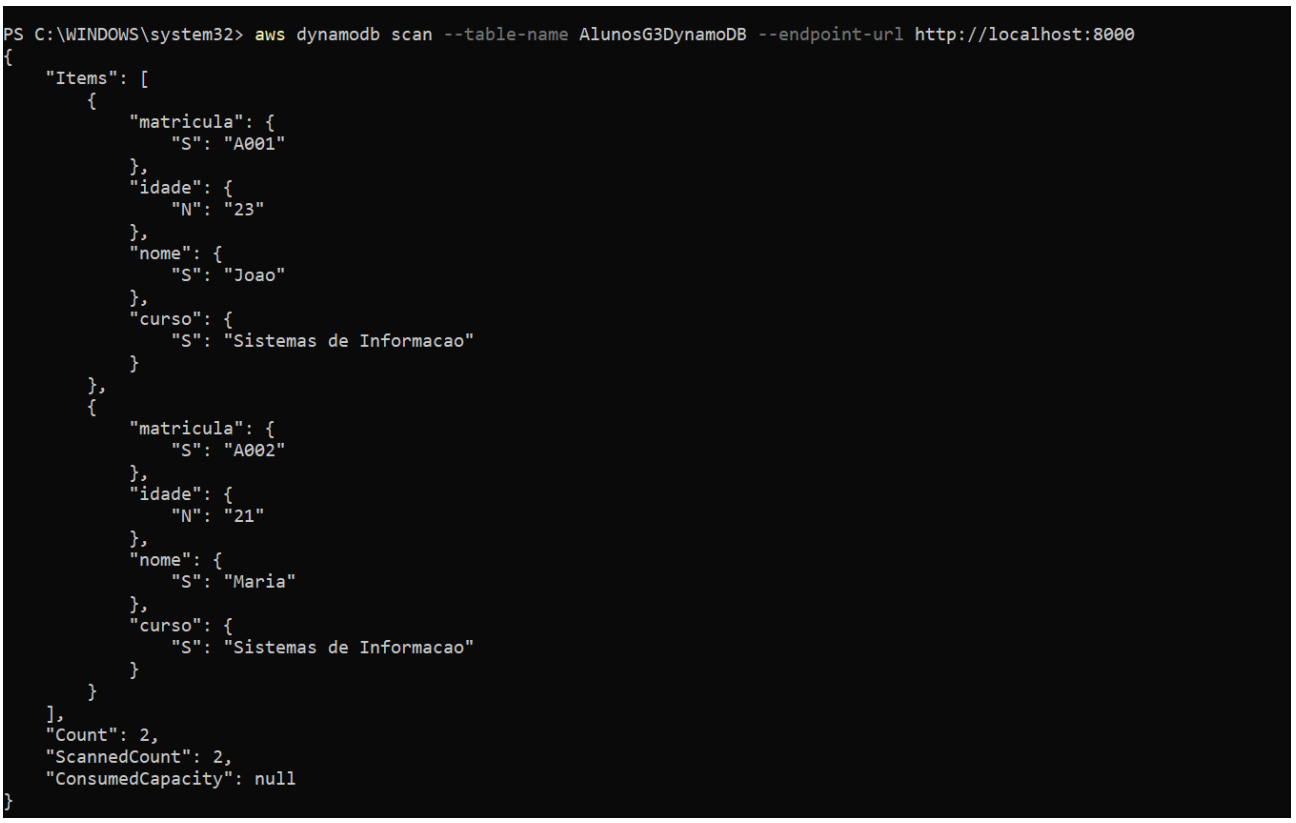
PS C:\WINDOWS\system32> aws dynamodb update-item --table-name AlunosG3DynamoDB --key file://chave-joao.json --update-expression "SET idade = :novaIdade" --expression-attribute-values file://atualizacao.json --return-values ALL_NEW --endpoint-url http://localhost:8000

```
{
    "Attributes": {
        "matricula": {
            "S": "A001"
        },
```

```
        "idade": {
            "N": "23"
        },
        "nome": {
            "S": "Joao"
        },
        "curso": {
            "S": "Sistemas de Informacao"
        }
    }
}
```


LISTANDO TODOS OS ALUNOS DO BANCO:

O comando `scan` foi utilizado para percorrer a tabela e listar os registros existentes. O resultado apresentou João, agora com 23 anos, e Maria, com 21 anos. Essa operação permitiu verificar os dados armazenados após a atualização.

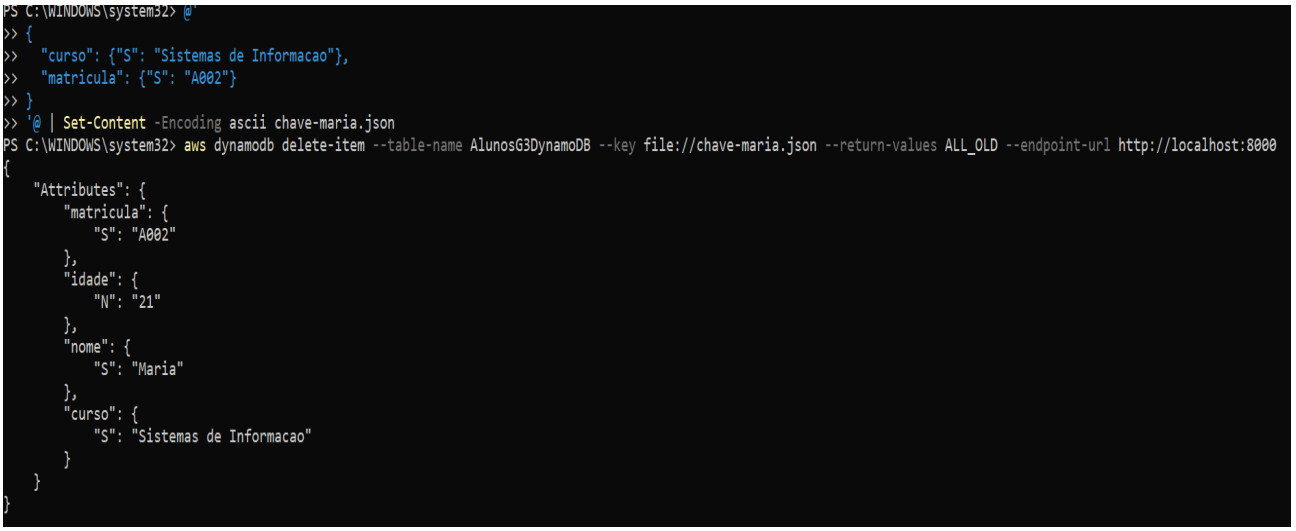


```
PS C:\WINDOWS\system32> aws dynamodb scan --table-name AlunosG3DynamoDB --endpoint-url http://localhost:8000
{
    "Items": [
        {
            "matricula": {
                "S": "A001"
            },
            "idade": {
                "N": "23"
            },
            "nome": {
                "S": "Joao"
            },
            "curso": {
                "S": "Sistemas de Informacao"
            }
        },
        {
            "matricula": {
                "S": "A002"
            },
            "idade": {
                "N": "21"
            },
            "nome": {
                "S": "Maria"
            },
            "curso": {
                "S": "Sistemas de Informacao"
            }
        }
    ],
    "Count": 2,
    "ScannedCount": 2,
    "ConsumedCapacity": null
}
```

```
PS C:\WINDOWS\system32> aws dynamodb scan --table-name AlunosG3DynamoDB
--endpoint-url http://localhost:8000
{
    "Items": [
        {
            "matricula": {
                "S": "A001"
            },
            "idade": {
                "N": "23"
            },
            "nome": {
                "S": "Joao"
            },
            "curso": {
                "S": "Sistemas de Informacao"
            }
        },
        {
            "matricula": {
                "S": "A002"
            },
            "idade": {
                "N": "21"
            },
            "nome": {
                "S": "Maria"
            },
            "curso": {
                "S": "Sistemas de Informacao"
            }
        }
    ],
    "Count": 2,
    "ScannedCount": 2,
    "ConsumedCapacity": null
}
```

DELETANDO A ALUNA MARIA:

O comando `delete-item` foi utilizado para remover Maria da tabela, identificando seu registro por meio das chaves presentes no arquivo `chave-maria.json`. A opção `--return-values ALL_OLD` solicitou que os dados do item fossem retornados antes da exclusão, permitindo confirmar qual registro foi removido.

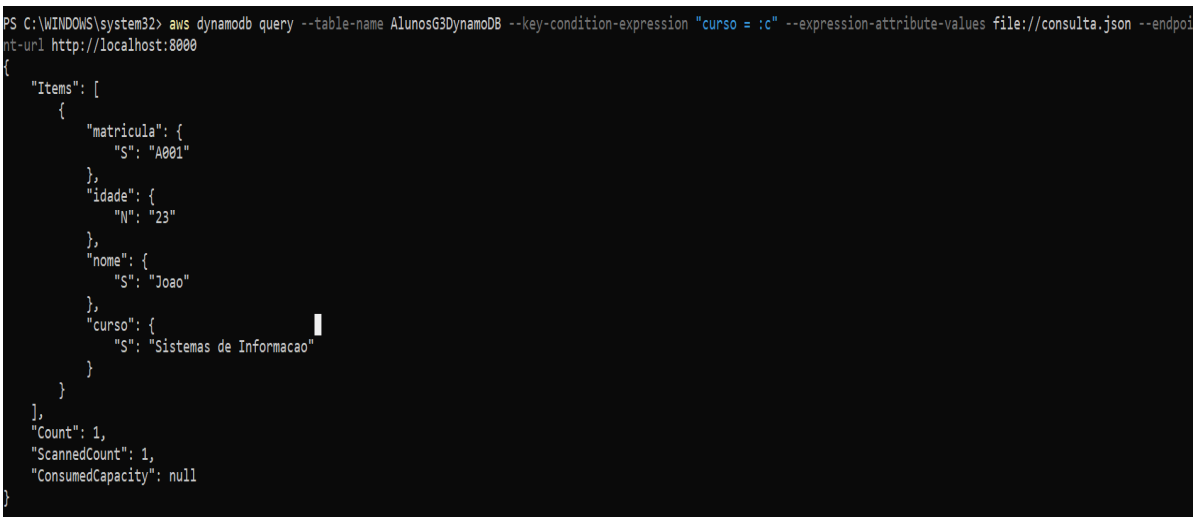


PS C:\WINDOWS\system32> aws dynamodb delete-item --table-name AlunosG3DynamoDB --key file://chave-maria.json --return-values ALL_OLD --endpoint-url http://localhost:8000


```
{
    "Attributes": {
        "matricula": {
            "S": "A002"
        },
        "idade": {
            "N": "21"
        },
        "nome": {
            "S": "Maria"
        },
        "curso": {
            "S": "Sistemas de Informacao"
        }
    }
}
```


LISTANDO O BANCO E MOSTRANDO Q SO RESTOU JOAO:

O comando `query` foi utilizado para consultar os alunos do curso de Sistemas de Informação. A expressão `curso = :c` definiu o critério de pesquisa, e o arquivo `consulta.json` forneceu o valor do curso. O resultado apresentou somente João e `Count: 1`, confirmando que a consulta encontrou um único registro após a exclusão de Maria.

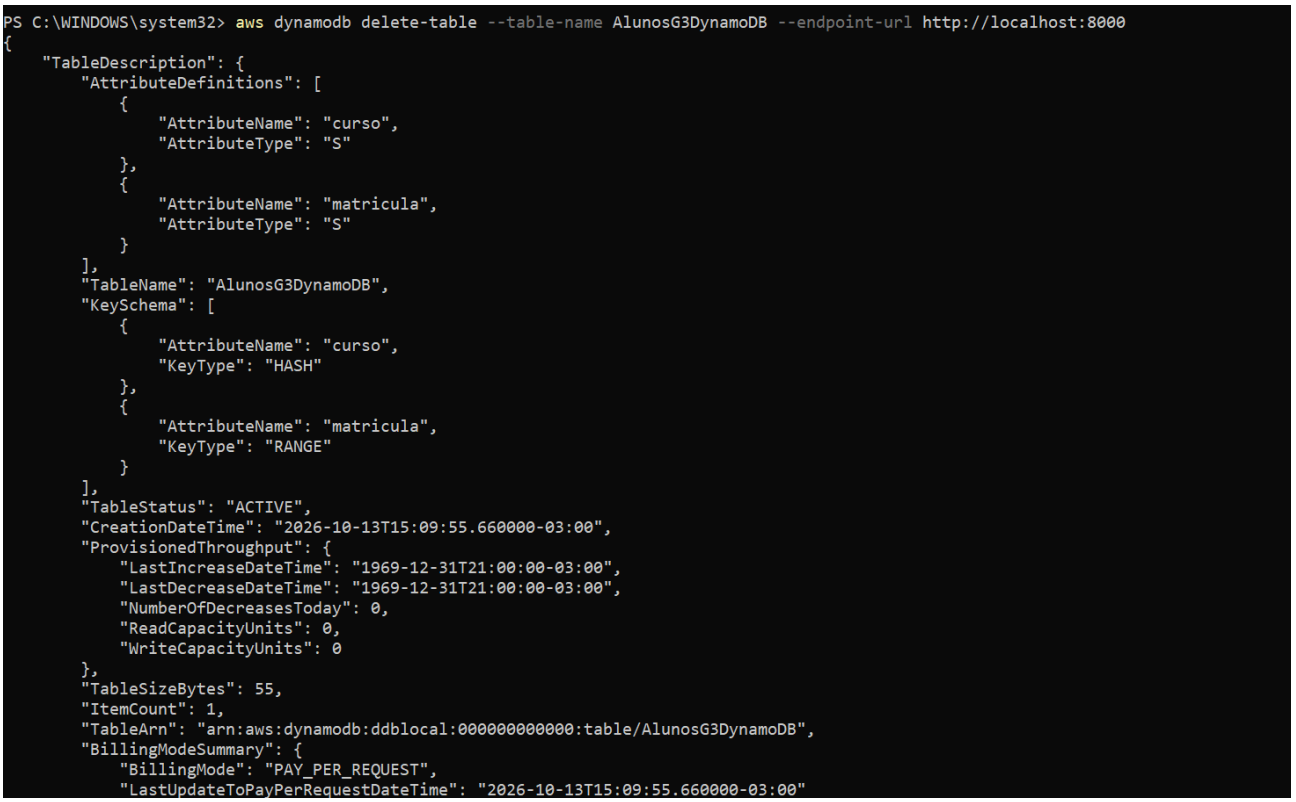


PS C:\WINDOWS\system32> aws dynamodb query --table-name AlunosG3DynamoDB --key-condition-expression "curso = :c" --expression-attribute-values file://consulta.json --endpoint-url http://localhost:8000
{
"Items": [
{
"matricula": {
"S": "A001"
},
"idade": {
"N": "23"
},
"nome": {
"S": "Joao"
},
"curso": {
"S": "Sistemas de Informacao"
}
}
],
"Count": 1,
"ScannedCount": 1,
"ConsumedCapacity": null

}

DELETANDO A TABELA:

O comando `aws dynamodb delete-table` foi utilizado para excluir a tabela `AlunosG3DynamoDB` após a conclusão dos testes. Essa etapa encerrou a prática, removendo a estrutura utilizada para armazenar os dados dos alunos.



Por fim, utilizamos o comando `aws dynamodb list-tables` para listar as tabelas existentes no DynamoDB Local. O resultado apresentou uma lista vazia, confirmando que a tabela criada durante a prática havia sido excluída e que não restaram tabelas nessa instância local.

Conclusão

O DynamoDB atende aplicações com acesso bem definido e grande escala. Sua escolha exige planejar chaves, índices e custos e avaliar as limitações das consultas. A demonstração com DynamoDB Local apresenta as operações fundamentais em um ambiente de desenvolvimento no próprio computador.

Referências

Fontes oficiais da Amazon Web Services. Acesso em: 13 out. 2026.

[1] AWS. What is Amazon DynamoDB?
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

[2] AWS. Core components of Amazon DynamoDB.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

[3] AWS. DynamoDB read consistency.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

[4] AWS. Amazon DynamoDB Transactions: How it works.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html

[5] STORMACQ, Sébastien. Happy 10th Birthday, DynamoDB! AWS News Blog, 18 fev. 2022.
https://aws.amazon.com/blogs/aws/happy-birthday-dynamodb/

[6] AWS. Amazon DynamoDB Customers.
https://aws.amazon.com/dynamodb/customers/

[7] AWS. First steps for modeling relational data in DynamoDB.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-modeling-nosql.html

[8] AWS. Constraints in Amazon DynamoDB.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Constraints.html

[9] AWS. DynamoDB - AWS CLI Command Reference.
https://docs.aws.amazon.com/cli/latest/reference/dynamodb/

[10] AWS. Getting started with AWS CloudShell.
https://docs.aws.amazon.com/cloudshell/latest/userguide/getting-started.html

[11] AWS. Query - AWS CLI Command Reference.
https://docs.aws.amazon.com/cli/latest/reference/dynamodb/query.html

[12] AWS. Amazon DynamoDB Features.
https://aws.amazon.com/dynamodb/features/

[13] AWS. Amazon DynamoDB Pricing.
https://aws.amazon.com/dynamodb/pricing/

[14] AWS. Put-item - AWS CLI Command Reference.
https://docs.aws.amazon.com/cli/latest/reference/dynamodb/put-item.html

[15] AWS. Update-item - AWS CLI Command Reference.
https://docs.aws.amazon.com/cli/latest/reference/dynamodb/update-item.html

[16] AWS. Scan - AWS CLI Command Reference.
https://docs.aws.amazon.com/cli/latest/reference/dynamodb/scan.html

[17] AWS. Setting up DynamoDB local (downloadable version).
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DynamoDBLocal.html